Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Lab on Asymmetric Criptography

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

RSA baseline

RSA is an asymmetric cryptographic algorithm based on the ***integer factorization problem***.

What we would like to have in principle:

RSA baseline

RSA is an asymmetric cryptographic algorithm based on the ***integer factorization problem***.

What we would like to have in principle:

1. (n, e) will be the public key, i.e., $k_{public} = (n, e)$
2. (n, d) will be the private key, i.e., $k_{private} = (n, d)$

RSA Key Generation

1. Choose two large prime numbers p and q

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

3. Compute the totient: $\Phi(n) = (p-1)(q-1)$

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

3. Compute the totient: $\Phi(n) = (p-1)(q-1)$

4. Choose an integer e such that:
   - must be prime
   - must be smaller than $\Phi(n)$, i.e., $1 < e < \phi(n)$
   - must not be a factor of $\Phi(n)$, i.e., must be coprime to $\phi(n)$
   - in practice:
     - we can use the Euclidean Algorithm to test if e is coprime to $\phi(n)$
     - most implementations use $e = 65537$ (after checking $\Phi(n)$ isn't divisible by 65537)

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

3. Compute the totient: $\Phi(n) = (p-1)(q-1)$

4. Choose an integer e such that:
   - must be prime
   - must be smaller than $\Phi(n)$, i.e., $1 < e < \phi(n)$
   - must not be a factor of $\Phi(n)$, i.e., must be coprime to $\phi(n)$
   - in practice:
     - we can use the Euclidean Algorithm to test if e is coprime to $\phi(n)$
     - most implementations use $e = 65537$ (after checking $\Phi(n)$ isn't divisible by 65537)

5. Choose an integer d such that:
   - The product $e \cdot d$ divided by $\Phi(n)$ has a remainder of 1, i.e., d is modular inverse of e modulo $\phi(n)$: $d \equiv e^{-1} \bmod \phi(n)$
   - in practice: we can use the Extended Euclidean algorithm to compute d

RSA: Core Functionality

RSA: Encryption

The encryption function is defined as:

$$y = E_{k_{public}}(x) = x^e \mod n$$

where x is the message, y is the ciphertext, (e, n) is the public key of the person <u>receiving</u> the message. Indeed, only the person with the private key (d, n) can decrypt the message.

RSA: Encryption

The encryption function is defined as:

$$y = E_{k_{public}}(x) = x^e \mod n$$

where x is the message, y is the ciphertext, (e, n) is the public key of the person <u>receiving</u> the message. Indeed, only the person with the private key (d, n) can decrypt the message.

WARNING: As it must be, RSA uses the public key of the receiver to encrypt the message!

RSA Decryption

The decryption function is defined as:

$$x = D_{k_{private}}(y) = y^d = (x^e)^d = x^{ed} = x^1 = x \mod n$$

RSA Decryption

The decryption function is defined as:

$$x = D_{k_{private}}(y) = y^d = (x^e)^d = x^{ed} = x^1 = x \mod n$$

where x is the message, y is the ciphertext, (d, n) is the private key of the person <u>receiving</u> the message.

WARNING: As it must be, RSA uses the private key of the receiver to decrypt the message.

RSA in Python

We perform now encryption and decryption:

```
# input message
x = 60
print("The input message is:", x)

# encryption
y = x ** e % n
print("The encrypted message is:", y)

# decryption
z = y ** d % n
print("The decrypted message is:", z)

# this must be true!
assert x == z
```

Converting strings to numbers and vice versa

RSA works with numbers but usually messages are text. Use this code to convert from number to string.

Converting strings to numbers and vice versa

RSA works with numbers but usually messages are text. Use this code to convert from number to string.

```
# m is the decrypted message

byte_len = (m.bit_length() + 7) // 8
flag = m.to_bytes(byte_len, 'big').decode('utf-8')

# flag is a string
```

Converting strings to numbers and vice versa

RSA works with numbers but usually messages are text. Use this code to convert from number to string.

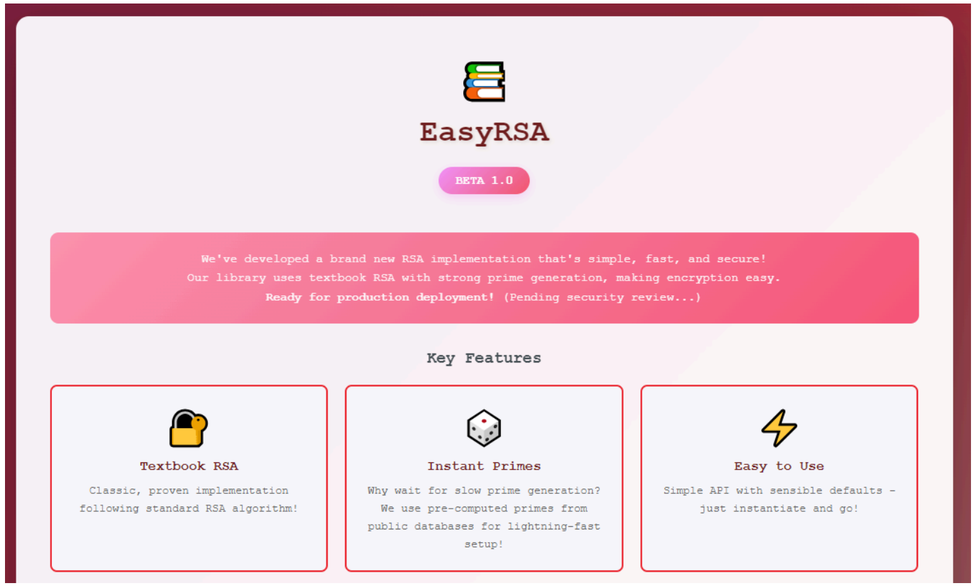
```
# m is the decrypted message

byte_len = (m.bit_length() + 7) // 8
flag = m.to_bytes(byte_len, 'big').decode('utf-8')

# flag is a string
```

```
# message is the string

data = message.encode('utf-8')
numeric_message = int.from_bytes(data, 'big')

# numeric_message is the number
```

EasyRSA 1.0

Crypto Challenge 08

EasyRSA 1.0 - Reconnaissance



What can we observe? The challenge...

- claims to implement **RSA** encryption;
- provides a file to be cracked;

EasyRSA 1.0 - Hypothesis

EasyRSA 1.0 - Hypothesis

We have n and we have e. To decrypt the message we need either p or q.

If we can factoryze n we should be able to decypher the message.

EasyRSA 1.0 - Hypothesis

We have n and we have e. To decrypt the message we need either p or q.

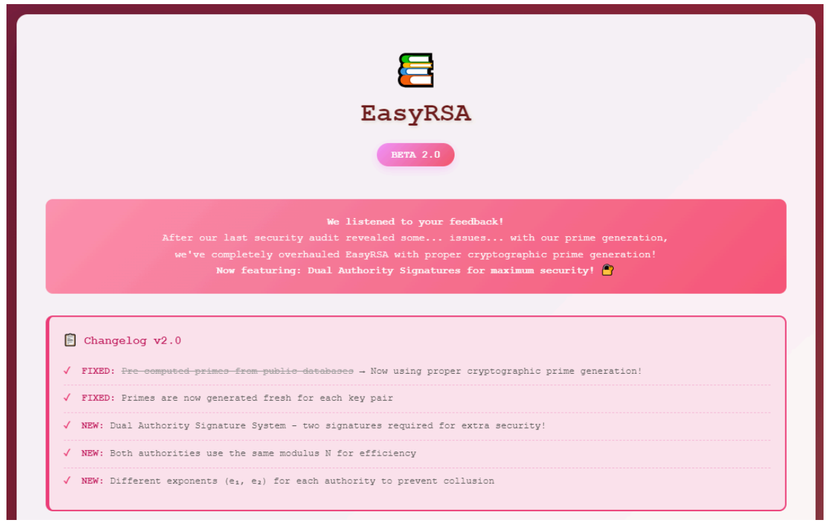
If we can factoryze n we should be able to decypher the message.

> The value of n seems too **short**. There are online some tools to factorize big umbers

EasyRSA 2.0

Crypto Challenge 09

EasyRSA 2.0 - Reconnaissance



What can we observe?

- A **secret document** get signed by two differen authorities.
- Each signature use a different exponent.
- Both signatures share the same **modulus** N.

EasyRSA 2.0 - Hypothesis

EasyRSA 2.0 - Hypothesis

1. The secret document is signed using the same RSA modulus using different exponents.

2. This signature is done encrypting the document with the **private** key so that everyone can decipher the document and validate the signature.

3. Since the goal is just obtain the secret document, we should treat it as it is normal encryption and focus on that.

> How to check if there is a way to decypher the document?

Satcom Intercept

Crypto Challenge 11

Satcom Intercept - Reconnaissance

What can we observe?

- There was a transmission that was repeted 5 times, one for each satellite.
- The encryption was done all with the **same exponent**.
- All the transmission used different **moduli**.
- The exponent used is 5.

Satcom Intercept - Hypothesis

The same plaintext is encrypted using RSA with a low public exponent across different moduli:

$$c_i = m^e \bmod n_i$$

If the same message is encrypted at least e times using:

- A small exponent,
- Different moduli,
- No padding,

the ciphertexts can be combined using the Chinese Remainder Theorem to recover:

$$m^e$$

over the integers.

Satcom Intercept - Hypothesis

The same plaintext is encrypted using RSA with a low public exponent across different moduli:

$$c_i = m^e \bmod n_i$$

If the same message is encrypted at least e times using:

- A small exponent,
- Different moduli,
- No padding,

the ciphertexts can be combined using the Chinese Remainder Theorem to recover:

$$m^e$$

over the integers.

> The low e with the same plaintext encrypted at least e times can be used to recover the encryption parameters.